

CSA15 Cloud Computing and Big Data Analytics

CO3 AT1 - Guided Cloud Access Experiments Workbook

Course Outcome	CO3 - Implement cloud-based solutions using cloud storage, web APIs, and platform services for real-world applications.
Unit Alignment	Unit 3 - Access to Cloud: cloud access, platforms, web applications, APIs, storage models, SAN, serverless, edge/fog, collaboration, security and development workflow.
Faculty	Dr C. Rajesh Babu, Professor, Department of CSE
Employee ID	SSETSCS415
Submission Mode	Individual PDF evidence through Google Classroom and GitHub repository link.

How This Workbook Works

Stage	Student Action
Learn	Read the short technical note before touching the tool.
Do	Perform the guided action using the recommended tool.
Observe	Check what changed on the screen or in the output.
Record	Paste screenshots, URLs, tables, command outputs or diagram references.
Explain	Write the technical reasoning in your own words.
Extend	Apply the same concept to the capstone product continued from CO2 AT3.
Submit	Upload the completed evidence to GitHub and Google Classroom.

Important Safety Rule

Screenshots must not reveal passwords, private keys, API tokens, payment details, personal account recovery information or sensitive user data. Crop or mask sensitive values before submission.

Student and Faculty Details

Student Name	T.PRAKEERTHI	Register Number	192524279
Class / Section	B.Tech (AI&DS)	Slot	B
Course	CSA15 Cloud Computing and Big Data Analytics	Assessment	CO3 AT1
Capstone Title	Medi Queue AI: Hospital of Queue and No-show. prediction App.	Submission Date	03/07/2026
Faculty	Dr C. Rajesh Babu	Employee ID	SSETSCS415
Department	Computer Science and Engineering	GitHub Link	https://github.com/tarigondaprakeerthi/CSA1512-Cloud-computing

Tool Links

Tool	Purpose	Link
Firebase Console	Cloud storage and project evidence	https://console.firebase.google.com/
Postman	REST API request-response evidence	https://www.postman.com/downloads/
JSONPlaceholder	Safe public API for GET/POST practice	https://jsonplaceholder.typicode.com/
ReqRes	Safe public API for request-response practice	https://reqres.in/
draw.io	Architecture and workflow diagrams	https://app.diagrams.net/
GitHub	Repository evidence submission	https://github.com/
GitHub Pages	Simple hosted web access evidence	https://pages.github.com/

Experiment 1: Cloud Storage Model Comparison and Selection

Original CO3 Question	Create and document a cloud storage comparison between object, block, and file storage. Select the best storage model for backups, VM disks, and shared enterprise documents.
-----------------------	---

Tools	Firebase Console, Firebase Storage, draw.io, browser, GitHub.
CO2 AT3 Continuity	Use the same capstone title and infrastructure direction already defended in CO2 AT3. Do not repeat VM sizing; focus on cloud service access.

Learn

A cloud application may store files, database records, VM disks, images, logs and shared documents. Object storage, block storage and file storage solve different workload problems. Firebase Storage is used here as a beginner-friendly object storage console because students can create a project, open a storage bucket, upload a file, observe metadata and reason about access control from the browser.

Do

- Begin in the browser at <https://console.firebase.google.com/>. Sign in with the account approved for academic work. After the console opens, use Add project or Create a project. Give the project a course-friendly name such as csa15-co3-regno. If Google Analytics is shown, keep it disabled unless faculty instructs otherwise. Create the project and wait until the project dashboard opens. The screen to notice is the Firebase project overview page.
- In the left navigation area, open Build and choose Storage. If Storage is not yet enabled, choose the option to get started. Select the default bucket location shown by Firebase or the nearest available region. Use test access only for lab observation if the console requires an initial rule choice, and remember that production apps must use strict authenticated rules. The screen to notice is the storage bucket page.
- Create a small text file on the computer named student_backup.txt. Add two lines inside it: the student register number and the capstone title. Return to Firebase Storage and use Upload file. Select student_backup.txt and wait until it appears in the bucket list. The screen to notice is the object list showing the uploaded file.
- Open the uploaded file entry. Observe the metadata area: file name, file size, content type, storage path, creation/update time and access status. If a download URL or access token is visible, do not paste sensitive token values in the workbook. Record only the safe evidence: filename, type, size and storage path.
- Open draw.io and draw a three-part comparison diagram: object storage for uploaded files/backups, block storage for VM disks, and file storage for shared folders. Under the diagram, write where your capstone will use storage and why Firebase Storage represents object storage in this experiment.

Observe

Check whether the uploaded file is visible, whether metadata is displayed, and whether public/private access is controlled. Do not expose sensitive files.

Record Evidence

Evidence Item	Student Reference / Screenshot Number	Checkpoint
Cloud storage dashboard screenshot.	Screenshot 1 – Cloud Storage Dashboard	Completed
Uploaded object/file screenshot.	Screenshot 2 – Patient Data File Uploaded	Completed
Metadata screenshot.	Screenshot 3 – File Metadata (Name, Size, Type, Last Modified)	Completed

Completed storage model comparison table.	Table 1 – Storage Model Comparison	Completed
Capstone storage recommendation table.	Table 2 – Medi Queue AI Storage Recommendation	Completed

Storage Model Comparison Table:

Storage Model	Advantages	Disadvantages	Suitable for MediQueue AI
Object Storage	Stores medical reports, prescriptions, and images; highly scalable	Not suitable for complex queries	✓ Yes
Block Storage	High performance for virtual machines	Expensive for large files	✗ No
File Storage	Easy shared access between applications	Limited scalability	△ Partially

Capstone Storage Recommendation Table

Application Data	Recommended Storage Reason	
Patient Reports (PDFs)	Object Storage	Stores large files securely and economically
Appointment Records	SQL Database	Fast search and structured data
Queue Information	SQL Database	Real-time updates and queries
No-Show Prediction Results	SQL Database	Stores AI prediction outputs
Medical Images	Object Storage	Efficient for large image files
Application Logs	Object Storage	Low-cost archival and monitoring

Explain

Why is one storage model not suitable for all workloads? Justify using backup, VM disk and shared-folder examples, then extend the answer to your capstone product.

Student Explanation Space

One storage model cannot handle every workload because different applications have different needs such as speed, cost, scalability, and sharing.

- Backup: Object Storage is best because it is low-cost and stores large backup files securely.
- VM Disk: Block Storage is best because it provides fast read/write speed for running virtual machines.
- Shared Folder: File Storage is best because multiple users can access and share files easily.

Medi Queue AI

For Medi Queue AI:

- Object Storage stores patient reports, medical images, and backups.
- Block Storage is used for the virtual machine running the application.
- File Storage stores shared hospital documents.

Conclusion: Using the right storage model for each workload improves performance, reliability, and cost efficiency.

Extend to Capstone

Select the storage model required for your capstone product: user uploads, app images, database attachments, logs, backups or shared documents.

Capstone Title	MediQueue AI – Hospital OP Queue & No-Show Prediction App
CO2 AT3 Infrastructure Decision Carried Forward	Used Azure Virtual Machine with Object Storage and SQL Database for reliable cloud hosting.
Cloud Service / Access Layer Added in this Experiment	Added Cloud Storage for patient reports and backups with secure cloud access.
Risk or Limitation Identified	Risk of unauthorized access and dependence on internet connectivity.
Improvement for Prototype or Pilot	Implement stronger security, automatic backups, and scalable storage for better performance.

Submit

GitHub Folder	CO3_AT1/Experiment_1
Files to Upload	Completed PDF, diagrams, screenshots and any exported table.
Google Classroom Entry	Paste GitHub repository link and attach final PDF.

Experiment 2: REST API Workflow for a Cloud-Based Student Portal

Original CO3 Question	Develop a simple REST API workflow for a cloud-based student portal. Show request, response, authentication, and deployment flow.
-----------------------	---

Tools	Postman Web/Desktop, JSONPlaceholder or ReqRes, draw.io, browser, GitHub.
CO2 AT3 Continuity	Use the same capstone title and infrastructure direction already defended in CO2 AT3. Do not repeat VM sizing; focus on cloud service access.

Learn

REST API means Representational State Transfer Application Programming Interface. For beginners, think of it as a controlled doorway through which a client app asks a cloud service for data or sends data to be processed. A browser or mobile app sends a request to an API endpoint. The API validates the request, applies business logic, communicates with storage or database services, and returns a structured response. Postman is used because it lets students send requests, view status codes, inspect headers, read JSON responses and test API behavior without writing a full app first.

Do

- Open <https://www.postman.com/> or the Postman desktop application. Sign in or create a free account if Postman asks for one. After the workspace opens, create a personal workspace or use the default workspace. Create a collection named CSA15_CO3_AT1_API so the evidence is organized.
- Open a new request tab. Keep the method as GET. Enter <https://jsonplaceholder.typicode.com/users> in the request URL field and send the request. Look at the response area. The expected observation is a successful status such as 200 OK and a JSON response containing user-like records.
- Read one object from the JSON response. Identify fields such as id, name, username, email, address and company. Record three fields in the workbook and explain what each field would mean in a student portal or capstone product.
- Create another request with method POST. Use <https://jsonplaceholder.typicode.com/posts> as the URL. Open the Body area, choose raw, select JSON, and enter a small JSON body such as {"title":"CSA15 cloud test","body":"API evidence","userId":1}. Send the request. The expected observation is a created/test response, often with status 201 or a simulated response.
- Open draw.io and draw the request-response flow. Show Browser/Mobile Client -> REST API Endpoint -> Backend Logic -> Database/Storage -> JSON Response. Add a small authentication gate before backend logic to show that real cloud applications should protect API access.

Observe

Verify whether the request returns status 200 or a valid test response. Identify JSON fields and explain what each field represents.

Record Evidence

Evidence Item	Student Reference / Screenshot Number	Checkpoint
Postman GET request screenshot.	Screenshot 1 – GET Request	Completed
Status code and response time screenshot.	Screenshot 2 – Status Code & Response Time	Completed

JSON field identification table.	Table 1 – JSON Fields	Completed
POST request body and response screenshot.	Screenshot 3 – POST Request & Response	Completed
API workflow diagram.	Figure 1 – API Workflow	Completed

Explain

Why should a cloud application expose data through APIs instead of allowing users to directly access the database?

Student Explanation Space

Cloud applications should use APIs instead of giving users direct database access because APIs provide security, control, and reliability. APIs authenticate users, validate requests, and return only the required data, preventing unauthorized access or accidental changes to the database. They also hide the database structure, making the application easier to maintain and update.

For MediQueue AI, patients and hospital staff access appointment details, queue status, and no-show predictions through secure APIs. The APIs communicate with the database and return only the necessary information, ensuring data security, privacy, and efficient application performance.

Extend to Capstone

Write two API endpoints for your capstone product and explain the input, backend action, storage access and response.

Capstone Title	MediQueue AI – Hospital OP Queue & No-Show Prediction App
CO2 AT3 Infrastructure Decision Carried Forward	Used Azure VM with SQL Database and Object Storage.
Cloud Service / Access Layer Added in this Experiment	REST API for secure data access.
Risk or Limitation Identified	Unauthorized API access if security is weak.
Improvement for Prototype or Pilot	Add API authentication, HTTPS encryption, and rate limiting.

Submit

GitHub Folder	CO3_AT1/Experiment_2
Files to Upload	Completed PDF, diagrams, screenshots and any exported table.
Google Classroom Entry	Paste GitHub repository link and attach final PDF.

Experiment 3: Serverless Event Flow for Image Upload Processing

Original CO3 Question	Design a serverless event flow for image upload processing using object storage, function-as-a-service, and notification service.
Tools	draw.io, Firebase Storage evidence from Experiment 1, optional Pipedream/Make simulation, GitHub.
CO2 AT3 Continuity	Use the same capstone title and infrastructure direction already defended in CO2 AT3. Do not repeat VM sizing; focus on cloud service access.

Learn

Serverless computing allows a cloud platform to run code in response to events without the student managing the

server directly. In this experiment, students do not need paid cloud function deployment. They design and document a correct event-driven flow. The key idea is: an event happens, a cloud function is triggered, the function performs a small task, and another cloud service records or sends the output.

Do

- Start with the uploaded file evidence from Experiment 1. Treat the upload as the event. For capstone mapping, choose one event such as image upload, booking creation, complaint submission, attendance scan, payment update or alert generation.
- In the workbook, write the event source in a short table. Mention who performs the action, what data is created, where it is stored, and what should happen automatically after the event.
- Design the cloud function behavior without writing production code. Name the function clearly, for example processUploadedImage, confirmBooking, validateAttendance or sendIssueAlert. Write what the function receives, what it checks, what it changes, and what output it produces.
- Decide the output service. It may be a notification, email, database status update, dashboard row, resized image, log entry or report update. Also decide what should happen if the function fails: retry, error log, faculty/admin alert or manual review.
- Draw the event flow in draw.io using this visual order: User Action -> Cloud Storage/Database Event -> Serverless Function -> Processing Decision -> Output Service -> Log/Monitoring. Paste the diagram and complete the trigger-action-output table.

Observe

Identify the trigger, processing function, output service, failure point and retry/alert requirement. If billing blocks real function deployment, submit a technically accurate simulation and architecture evidence.

Record Evidence

Evidence Item	Student Reference / Screenshot Number	Checkpoint

Event source evidence or simulated event table.	Table 1 – Event Source	Completed
Serverless flow diagram.	Figure 1 – Serverless Workflow	Completed
Trigger-action-output table.	Table 2 – Trigger–Action–Output	Completed
Failure and retry note.	Note 1 – Failure & Retry	Completed
Capstone event mapping.	Table 3 – MediQueue AI Event Mapping	Completed

JSON Field Identification Table

JSON Field	Description
patientId	Unique patient ID
patientName	Patient name
appointmentDate	Appointment date
queueNumber	Queue position
prediction	No-show prediction result

API Workflow Diagram

```

Client (Mobile App/Web)
↓
GET/POST Request (Postman)
↓
API Server
↓
SQL Database
↓
JSON Response
↓
Client Displays Queue & Prediction
  
```

Explain

Why is serverless suitable for upload/event processing? Mention one limitation also.

Student Explanation Space:

Serverless is suitable for upload and event processing because it automatically runs code whenever an event occurs, such as a file upload or appointment booking. It scales automatically, reduces server management, and users pay only when the function executes.

Limitation: Serverless functions may experience cold start delays when they are invoked after being idle, which can slightly increase response time.

Extend to Capstone

Apply the serverless idea to one real capstone workflow, such as image processing, booking confirmation, attendance validation, alert generation or report update.

Capstone Title	MediQueue AI – Hospital OP Queue & No-Show Prediction App
CO2 AT3 Infrastructure Decision Carried Forward	Used Azure Virtual Machine, SQL Database, and Object Storage.
Cloud Service / Access Layer Added in this Experiment	Added Serverless Functions for event processing and notifications.
Risk or Limitation Identified	Cold start delay and dependence on cloud services.
Improvement for Prototype or Pilot	Optimize functions, improve monitoring, and add automatic retry with stronger security.

Submit

GitHub Folder	CO3_AT1/Experiment_3
Files to Upload	Completed PDF, diagrams, screenshots and any exported table.
Google Classroom Entry	Paste GitHub repository link and attach final PDF.

Experiment 4: Edge and Fog Computing for Smart Traffic Management

Original CO3 Question	Compare edge computing and fog computing for a smart traffic management system. Include architecture, latency considerations, and deployment location.
Tools	draw.io, Google Sheets or Excel latency table, optional Wokwi/IoT screenshot only if faculty permits, GitHub.
CO2 AT3 Continuity	Use the same capstone title and infrastructure direction already defended in CO2 AT3. Do not repeat VM sizing; focus on cloud service access.

Learn

Cloud computing is powerful but may be physically distant from sensors and users. Edge computing places small decisions very near the data source. Fog computing introduces an intermediate distributed layer between edge devices and the central cloud. This experiment is an architecture reasoning activity; students are not required to build a real traffic sensor. The correct output is a placement decision supported by latency and data-volume reasoning.

Do

- Use this scenario: traffic cameras and road sensors produce vehicle count, speed, congestion level and emergency vehicle alerts. Some actions must happen immediately at the signal, some can be handled at a regional traffic control unit, and some can be stored in the central cloud for analysis.
- Classify each task into edge, fog or cloud. Put instant signal switching and emergency alert detection near edge. Put junction-level aggregation and area coordination in fog. Put long-term storage, analytics dashboard and city-level reports in cloud.
- Open a spreadsheet and create a latency table with columns: task, data generated, decision deadline, suitable layer, reason. Use example decision deadlines such as 1 second for emergency signal response, 5-10 seconds for regional coordination and minutes/hours for reports. The exact numbers are reasoning aids, not measured network results.
- Draw the architecture in draw.io. Show sensors/cameras at the road, edge device at signal controller, fog node at regional traffic control, cloud platform for storage/analytics and dashboard for administrator. Use arrows to show data movement and labels to show why each layer is used.
- Connect the same idea to the capstone. If the capstone has live location, sensor, mobile offline, queue, emergency, QR attendance or fast alert behavior, identify what can be edge/fog/cloud. If the capstone does not need edge/fog, state that browser/mobile client -> cloud service access is sufficient and justify it.

Observe

Check whether time-critical actions are placed closer to data sources and whether long-term storage/analytics remain in the cloud.

Record Evidence

Evidence Item	Student Reference / Screenshot Number	Checkpoint
Task classification table.	Table 1 – Task Classification	Completed
Latency reasoning table.	Table 2 – Latency Analysis	Completed
Edge-fog-cloud architecture diagram.	Figure 1 – Edge–Fog–Cloud Architecture	Completed
Deployment location justification.	Table 3 – Deployment Justification	Completed
Capstone relevance note.	Note 1 – MediQueue AI Relevance	Completed

Task Classification Table

Task	Location
Patient data entry	Edge
Queue processing	Fog
Database & AI prediction	Cloud

Latency Reasoning Table

Task	Reason
Appointment booking	Low latency required
Queue updates	Fast local processing
AI prediction	High computing power in cloud

Deployment Location Justification

Component	Deployment Reason	
Mobile App	Edge	Quick user interaction
Hospital Gateway	Fog	Reduces response time
SQL Database & AI Cloud		Scalable and reliable

Explain

Which part of the system should run at edge, fog and cloud? Justify with latency and data-volume reasoning.

Student Explanation Space

For MediQueue AI – Hospital OP Queue & No-Show Prediction App:

- Edge: The mobile app/web app should run at the edge because patient registration and appointment booking require low latency and an immediate response. Only a small amount of data is processed.
- Fog: The hospital gateway/local server should run at the fog layer to handle queue updates, notifications, and local processing. This reduces network delay and avoids sending every request to the cloud.
- Cloud: The SQL database, AI no-show prediction, and backup storage should run in the cloud because they process large volumes of data and require high computing power, scalability, and long-term storage.

Justification:

The edge provides fast user interaction with minimal data processing, the fog reduces latency by handling local operations, and the cloud manages heavy computation and large data storage efficiently. This combination improves performance, scalability, and reliability for MediQueue AI.

Extend to Capstone

If your capstone has location-sensitive, sensor-based, offline-first or fast-alert requirements, identify the edge/fog/cloud placement. If not applicable, justify why direct cloud access is sufficient.

Capstone Title	MediQueue AI – Hospital OP Queue & No-Show Prediction App
CO2 AT3 Infrastructure Decision Carried Forward	Used Azure VM, SQL Database, and Object Storage.
Cloud Service / Access Layer Added in this Experiment	Added Edge–Fog–Cloud architecture for faster processing.
Risk or Limitation Identified	Network delay or connectivity issues between edge, fog, and cloud.
Improvement for Prototype or Pilot	Optimize data routing, improve monitoring, and enhance security for better performance.

Submit

GitHub Folder	CO3_AT1/Experiment_4
Files to Upload	Completed PDF, diagrams, screenshots and any exported table.
Google Classroom Entry	Paste GitHub repository link and attach final PDF.

Experiment 5: Cloud Application Deployment and Access Workflow

Original CO3 Question	Demonstrate a cloud application deployment workflow showing client access, browser interaction, web API call, storage access, and monitoring.
Tools	GitHub Pages as primary beginner path, Firebase Hosting as optional path, Postman, draw.io, GitHub repository, browser.
CO2 AT3 Continuity	Use the same capstone title and infrastructure direction already defended in CO2 AT3. Do not repeat VM sizing; focus on cloud service access.

Learn

A cloud-backed application is accessed through a browser or mobile client. The user action travels through frontend, API/backend, authentication, platform services, storage/database and monitoring/logging components. GitHub Pages is the primary beginner path because it can host a simple static page from a repository. Firebase Hosting may be used when the student is comfortable with Firebase CLI or Cloud Shell.

Do

- ☐ Create or open a GitHub repository for CO3 AT1. Add a simple index.html file. The page should show the capstone title, one user action and one cloud service used. Example page sections: product name, user action, API endpoint idea, storage used and evidence link.
- ☐ In the repository settings, open Pages. Select the main branch and root folder as the publishing source when available. Save the Pages setting and wait for GitHub to publish the site. Open the generated Pages URL in the browser and verify that the page loads.

- If Firebase Hosting is used instead, open Firebase Console, choose Hosting, follow the hosting setup guidance, and deploy using Firebase CLI or Cloud Shell only when faculty confirms that the device and account are ready. For most beginners, GitHub Pages evidence is sufficient for this experiment.
- Use Postman evidence from Experiment 2 or create a new test request that represents one action from the hosted page. Example: the user clicks Submit Complaint, the frontend sends POST /reports, the backend stores the complaint image path and returns a status response.
- Draw the complete access workflow in draw.io: Browser/Mobile Client -> Hosted Frontend -> API Endpoint -> Authentication Check -> Backend/Serverless Logic -> Cloud Storage/Database -> Response -> Monitoring/Log. Paste the hosted URL, browser screenshot, workflow diagram and GitHub repository link.

Observe

Verify whether the hosted page or simulated access URL opens, whether the API/storage flow is technically consistent, and whether monitoring/logging evidence is identified.

Record Evidence

Evidence Item	Student Reference / Screenshot Number	Checkpoint
Hosted page URL or approved simulation evidence.	Screenshot 1 – Hosted Page / Simulation	Completed
Browser access screenshot.	Screenshot 2 – Browser Access	Completed
API/storage workflow diagram.	Figure 1 – API & Storage Workflow	Completed
Monitoring/logging metric table.	Table 1 – Monitoring Metrics	Completed
GitHub repository link.	GitHub Repository – MediQueue AI	Completed

Monitoring/Logging Metric Table

Metric	Purpose
Response Time	Measures API speed
CPU Usage	Monitors server performance
Memory Usage	Tracks application memory
Error Rate	Detects API failures
Request Count	Monitors application traffic

Explain

Explain the complete request flow from client action to cloud output and monitoring.

Student Explanation Space

Complete Request Flow from Client to Cloud Output and Monitoring

1. Client Action: The patient books an appointment using the MediQueue AI mobile app or web application.
2. API Request: The request is sent securely to the cloud through a REST API.
3. Backend Processing: The cloud server validates the request and processes the appointment.
4. Storage Access: The application reads from or writes data to the SQL Database and stores files in Object Storage if required.
5. Cloud Output: The server returns a JSON response containing the appointment confirmation, queue number, and estimated waiting time.
6. Monitoring: Cloud monitoring services record response time, request count, CPU usage, memory usage, and errors to ensure the application performs reliably and any issues are detected quickly.

Extend to Capstone

Extend the workflow to the capstone product carried forward from CO2 AT3, showing how the planned infrastructure now exposes usable cloud services.

Capstone Title	MediQueue AI – Hospital OP Queue & No-Show Prediction App
CO2 AT3 Infrastructure Decision Carried Forward	Used Azure Virtual Machine, SQL Database, and Object Storage.
Cloud Service / Access Layer Added in this Experiment	Added cloud deployment with REST API and monitoring services
Risk or Limitation Identified	Network downtime or server failures may affect application availability.
Improvement for Prototype or Pilot	Implement auto-scaling, load balancing, regular monitoring, and automated backups for higher reliability.

Submit

GitHub Folder	CO3_AT1/Experiment_5
Files to Upload	Completed PDF, diagrams, screenshots and any exported table.
Google Classroom Entry	Paste GitHub repository link and attach final PDF.

Final Submission Checklist

Checklist Item	Status
Experiment 1 completed with storage model evidence	Completed
Experiment 2 completed with API evidence	Completed
Experiment 3 completed with serverless/event-flow evidence	Completed
Experiment 4 completed with edge/fog/cloud reasoning	Completed
Experiment 5 completed with cloud application access workflow	Completed
All diagrams exported and pasted	Completed
All screenshots are readable and privacy-safe	Completed
Capstone extension completed for every experiment	Completed
GitHub folder created and link added	Completed
Google Classroom submission completed	Completed

Student Assurance

Declaration	I confirm that this CO3 AT1 guided experiment workbook contains my own cloud access evidence, diagrams, observations, explanations and capstone extensions. I have not shared private credentials or sensitive information.
Student Signature	T.Prakeerthi
Faculty Verification	
Date	